

PROJECT REPORT

SMART CAMPUS COMPLAINT & MAINTENANCE MANAGEMENT SYSTEM

A Core Java Console-Based Application

Submitted in partial fulfillment of the academic requirements
for the course Programming in Java

Submitted By

Divyansh Verma

Registration No.: 25BAI11232

Academic Year: 2026–2027

Department of Computer Science and Engineering

VIT Bhopal University

Table of Contents

1. Introduction
 2. Problem Statement
 3. Objectives and Scope
 4. Functional Requirements
 5. Non-Functional Requirements
 6. System Design and Architecture
 7. Technologies and Tools Used
 8. Implementation
 9. Testing and Test Cases
 10. Results and Screenshots
 11. Limitations and Future Scope
 12. Conclusion
- References

1. Introduction

The Smart Campus Complaint & Maintenance Management System is a lightweight Java application designed to organize the reporting and handling of campus infrastructure issues. The system provides a centralized workflow in which students can submit complaints, administrators can review and assign them, and maintenance workers can view assigned work and mark issues as resolved.

The application is implemented entirely in Core Java and operates through a console interface. It uses Java Collections for managing complaint data and Java file I/O streams for persistent storage. The application also uses basic Java multithreading to perform background saving when the program exits.

The project focuses on a clear complaint lifecycle: an issue is created, reviewed and assigned, and finally resolved. This provides a simple form of accountability and makes complaint status visible to the appropriate users.

The project documentation identifies three principal roles—Student, Admin, and Maintenance staff—and defines role-specific workflows for each of them. □ filecite□turn0file0□L6-L12□

2. Problem Statement

University campuses contain classrooms, electrical fixtures, plumbing facilities, network equipment, air-conditioning systems, and other infrastructure that require regular maintenance. When problems are reported through scattered emails, verbal communication, or paper registers, complaints can be delayed, misplaced, or difficult to track.

The project statement identifies the need for a centralized system that can bring order to the complaint process, improve accountability, and allow reported issues to move through a defined lifecycle. □ filecite□turn0file1□L5-L7□

The proposed system addresses this problem by creating complaint tickets with identifiers, assigning tickets to maintenance personnel, tracking their status, and preserving complaint data across application sessions.

3. Objectives and Scope

3.1 Objectives

- Provide a centralized mechanism for reporting campus infrastructure complaints.
- Provide role-based access for students, administrators, and maintenance workers.
- Allow administrators to review complaints and assign them to maintenance workers.
- Allow maintenance workers to view assigned tasks and mark completed work as resolved.
- Maintain complaint information after application restart through persistent file storage.
- Provide a simple console-based interface suitable for a Core Java academic project.

3.2 Scope

The scope is intentionally focused on issue tracking. It includes secure role-based login workflows, centralized complaint creation and management, assignment and resolution, and persistent historical complaint storage. □filecite□turn0file1□L8-L13□

The system does not attempt to provide broader facilities-management functions such as payroll, inventory purchasing, vendor contracting, or other enterprise facilities operations.

3.3 Target Users

- Students / Staff – report infrastructure problems experienced on campus.
- Administrators – monitor incoming complaints and assign them to appropriate maintenance personnel.
- Maintenance Workers – view assigned tasks and resolve issues physically.

These user groups and responsibilities are explicitly defined in the project statement.

□filecite□turn0file1□L15-L18□

4. Functional Requirements

The functional requirements describe what the system must allow each user role to accomplish.

ID	Requirement	Actor	Expected Behaviour
FR-01	Role-based login	Student/Admin/Maintenance	User selects the appropriate role and logs into the corresponding workflow.
FR-02	Create complaint	Student	Student enters a complaint title and description and submits it.
FR-03	Generate complaint ID	System	A complaint receives an auto-incrementing identifier.
FR-04	View complaints	Admin	Administrator views the reported complaints and identifies tickets for action.
FR-05	Assign complaint	Admin	Administrator assigns a complaint to a maintenance username.
FR-06	View assigned work	Maintenance	Maintenance worker views complaints assigned to their account.
FR-07	Resolve complaint	Maintenance	Maintenance worker changes an assigned complaint to

FR-08	Persist data	System	RESOLVED. Complaint data is saved to and loaded from a binary .dat file.
FR-09	Exit and save	System	Application triggers background saving when the program exits.

The documented lifecycle of a complaint is OPEN → ASSIGNED → RESOLVED.

□ filecite □ turn0file1 □ L20-L24 □

5. Non-Functional Requirements

Usability: The console menus should be simple and understandable so that each role can complete its primary task without unnecessary navigation.

Reliability: Complaint information should remain available after the application is restarted through persistent file-based storage.

Maintainability: The project should use standard Core Java features and avoid unnecessary external dependencies.

Portability: The application should be runnable on a system with a Java Development Kit installed.

Performance: The application is intended to be lightweight and suitable for a small-to-moderate set of campus complaints.

Data Integrity: Complaint records should preserve their identifiers, descriptions, assignments, and status during save/load operations.

6. System Design and Architecture

6.1 High-Level Architecture

The application follows a simple console-based application architecture. The user interacts with the Java console interface, role-specific operations manipulate complaint data held in Java collections, and Java serialization provides persistence to a binary data file.

[SCREENSHOT PLACEHOLDER]

Figure 1: High-Level System Architecture

Insert an architecture diagram showing User Roles → Console Application → Complaint Management → File Persistence.

6.2 User Roles and Workflow

The main workflow is role-driven. A Student creates a complaint. An Admin reviews the complaint and assigns it to a maintenance worker. The Maintenance user then views the assigned task and

marks it as resolved. This workflow corresponds to the documented test procedure.

□ filecite□turn0file0□L35-L44□

[SCREENSHOT PLACEHOLDER]

Figure 2: Complaint Lifecycle

Insert a simple flow diagram: OPEN → ASSIGNED → RESOLVED.

6.3 Data Model

The available project documentation specifies a centralized complaint list, auto-incrementing complaint IDs, descriptions, assignment to maintenance usernames, status transitions, and persistence in complaints.dat. □ filecite□turn0file1□L20-L24□

A representative complaint record therefore contains the information required to support the workflow: complaint ID, title, description, assignment information, and current status. The exact internal class fields should be taken from the submitted source code if a code-level appendix is added.

7. Technologies and Tools Used

Technology / Tool	Use in Project
Programming Language	Core Java / JDK
Data Structures	Java Collections Framework – ArrayList and List
Persistence	Java File I/O – ObjectOutputStream and ObjectInputStream
Storage Format	Serialized binary .dat file
Development Environment	Visual Studio Code
Build Approach	Direct javac compilation; no Maven or external build tool required
Execution	Console-based Java application

The README identifies Core Java, ArrayList/List, Java serialization streams, and VS Code as the primary technologies and tools. □ filecite□turn0file0□L14-L18□

8. Implementation

8.1 Login and Role Selection

At application start, the user selects a login role. The documented roles are Student, Admin, and Maintenance. Each role is then provided with the operations relevant to its responsibilities.

[SCREENSHOT PLACEHOLDER]

Figure 3: Login / Role Selection Screen

Insert the console screenshot showing the login menu and role choices.

8.2 Student Complaint Logging

The Student workflow provides an option to create a new complaint. The user supplies a title and description, after which the complaint is added to the centralized complaint collection and receives an identifier.

[SCREENSHOT PLACEHOLDER]

Figure 4: Student Complaint Creation

Insert the console screenshot showing the complaint title/description input and generated complaint ID.

8.3 Administrator Assignment

The Admin workflow allows the administrator to view reported complaints and assign a selected complaint to a maintenance worker using the worker's username.

[SCREENSHOT PLACEHOLDER]

Figure 5: Admin Complaint Assignment

Insert the console screenshot showing complaint listing and assignment to a maintenance username.

8.4 Maintenance Resolution

The Maintenance workflow allows a worker to view tasks assigned to the worker's account and mark an appropriate task as resolved. The documented lifecycle is OPEN → ASSIGNED → RESOLVED.

[SCREENSHOT PLACEHOLDER]

Figure 6: Maintenance Task Resolution

Insert the console screenshot showing assigned complaints and the RESOLVED update.

8.5 Persistent Storage and Auto-Saving

Complaint data is stored using Java object serialization. The application reads and writes the complaint data through `ObjectInputStream` and `ObjectOutputStream`. The README also specifies background auto-saving through basic Java multithreading when the application exits.

□ filecite □ turn0file0 □ L11-L12 □

[SCREENSHOT PLACEHOLDER]

Figure 7: Data Persistence / Successful Relaunch

Insert a screenshot demonstrating that complaint data remains available after restarting the application.

9. Testing and Test Cases

The supplied README provides an end-to-end testing sequence covering complaint creation, administrative assignment, maintenance resolution, exit-time saving, and data recovery after relaunch. [filecite\turn0file0\L35-L45](#)

ID	Test	Input/Action	Expected Result	Result
TC-01	Student login	Student role selected; valid example username such as alice	Student menu is displayed	Pass / verify
TC-02	Create complaint	Title: Broken AC in Room 101; description supplied	Complaint created with an ID	Pass / verify
TC-03	Admin login and viewing	Admin role; username admin	All reported complaints are displayed	Pass / verify
TC-04	Assign complaint	Complaint ID and maintenance username bob	Complaint becomes assigned	Pass / verify
TC-05	Maintenance login	Maintenance role; username bob	Assigned complaint is visible	Pass / verify
TC-06	Resolve complaint	Select assigned complaint and choose resolve	Status becomes RESOLVED	Pass / verify
TC-07	Exit/save	Exit using login-menu choice 4	Background save is triggered	Pass / verify
TC-08	Persistence	Relaunch application	Previous complaint data is preserved	Pass / verify

9.1 Installation and Execution

1. Install the Java Development Kit (JDK).
2. Open the smart-campus-system project in Visual Studio Code.
3. Open a terminal inside the src/campus directory.
4. Compile using: `javac *.java`
5. Run using: `java Main`

These installation and execution steps are specified in the project README.

[filecite\turn0file0\L20-L33](#)

10. Results and Screenshots

The expected result is a working complaint-management workflow in which complaints can be created, assigned, resolved, and preserved between sessions. The supplied project documentation states that after relaunch, complaint data is preserved through the persistent storage mechanism.

[filecite\turn0file0\L43-L45](#)

Insert the following screenshots from the actual running application before final submission. Avoid adding screenshots that do not correspond to the implemented application.

- Login / role selection
- Student menu and complaint creation
- Generated complaint ID
- Admin complaint list
- Admin assignment operation
- Maintenance assigned-task view
- Complaint marked RESOLVED
- Application exit and save
- Relaunch showing preserved complaint data

[SCREENSHOT PLACEHOLDER]

Figure 8: Final Application Output

Insert the most representative final console output here.

11. Limitations and Future Scope

11.1 Limitations

- The current system is console-based rather than graphical or web-based.
- Storage is file-based serialization rather than a database-backed solution.
- The project scope is limited to complaint tracking and does not include inventory, procurement, payroll, or vendor management.
- The available project documentation does not specify advanced notifications, analytics dashboards, attachments, or mobile access.

11.2 Future Scope

- Develop a graphical desktop interface or web/mobile interface.
- Replace binary file persistence with a relational database such as MySQL or PostgreSQL.
- Add email or in-application notifications for assignment and resolution events.
- Add complaint categories, priorities, locations, timestamps, and service-level deadlines.
- Provide dashboards and reports for administrators to analyse complaint trends.
- Add authentication improvements and stronger access-control mechanisms.
- Allow image attachments for infrastructure problems.
- Add search, filtering, and historical complaint analytics.

These future enhancements extend the existing issue-tracking scope without changing the project's central complaint lifecycle.

12. Conclusion

The Smart Campus Complaint & Maintenance Management System provides a focused solution to the problem of unstructured campus infrastructure complaint handling. By introducing role-based workflows, centralized complaint records, assignment, status tracking, and persistent storage, the application establishes a clear path from issue reporting to resolution.

The project demonstrates practical use of Core Java concepts including collections, file I/O, object serialization, role-oriented program flow, and basic multithreading. Its lightweight console-based design keeps the implementation appropriate for an academic Java project while still demonstrating a complete end-to-end application workflow.

The documented lifecycle—OPEN → ASSIGNED → RESOLVED—provides a simple but effective model for accountability. With future integration of a database, graphical interface, notifications, and analytics, the same core concept could be extended into a more comprehensive campus maintenance platform.

References

1. Project README: “Smart Campus Complaint & Maintenance Management System” — supplied project documentation. [filecite\turn0file0\L3-L18](#)
2. Project Problem Statement: “Smart Campus Complaint & Maintenance Management System” — supplied project documentation. [filecite\turn0file1\L3-L24](#)
3. Java Development Kit (JDK) and standard Core Java APIs — used as specified by the supplied project documentation.